

## Assignment description

# Assignment 1

Solve the following exercises. Any non-code part of the assignment must be handed in as comments in the source code.

You write multiline comments between `(* and *)` and single-line comments using `//`.

Some of the exercises (which are marked) are copied more or less verbatim from the course book for your convenience, possibly with minor

[Home](#) > [Functional Programming, SWU \(Spring 2026\)](#) > [Assignment 1](#) 

## Assignment 1

Due 2 months ago · Locked a month ago · [Grades published](#)

Here, there are a few important things to note:

- The module title must be Assignment1 or the project will not compile
- All functions have a placeholder with a `failwith "not completed"`. This placeholder must be kept for all of the assignments that you have not solved or the project will not compile.
- This is a course in functional programming so any use of mutable variables or loops is strictly prohibited unless explicitly allowed, which is not the case for this assignment.

Ask the TAs for help with the setups if you need to.

## Green Exercises

|                                                                                       |                                                                                                                  |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
|  | <a href="#">Assignment</a>  |
|                                                                                       | <a href="#">Home</a>                                                                                             |
|                                                                                       | <a href="#">Assignment description</a>                                                                           |
|                                                                                       | <a href="#">Rubric</a>                                                                                           |
|                                                                                       | <a href="#">Course feedback</a>                                                                                  |

## Exercise 1.1

Write a function `sqr : int -> int` that given an integer `x` returns `x` squared

## Exercise 1.2

Write a function `pow : float -> float -> float` that given two floating point numbers `x` and `n` returns `x` to the power of `n`

**Hint:** Use the library function: `System.Math.Pow`

## Exercise 1.3 (based on 1.4 from HR)

Write a recursive function `sum : int -> int` such that given an integer `n` such that  $n \geq 0$  returns the sum of all integers from `0` to `n` inclusive.

$$\text{sum } n = 0 + 1 + 2 + 3 + \dots + n$$

**Hint:** Use two clauses with `0` and `n` as patterns.

## Exercise 1.4 (based on 1.5 from HR)

The sequence  $F_0, F_1, F_2, \dots$  of Fibonacci numbers is defined by:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \end{aligned}$$

Thus, the first members of the sequence are `0, 1, 1, 2, 3, 5, 8, 13, ...`

Write a function `fib : int -> int` that given an integer `n` such that  $n \geq 0$  computes  $F_n$ . Use a declaration with three clauses, where the patterns correspond to the three cases of the above

definition.

## Exercise 1.5

Write a function `dup : string -> string` that given a string `s` concatenates `s` with itself. You can use `+` to concatenate strings.

**Example:** `dup "Hi " = "Hi Hi "`

## Exercise 1.6

Write a function `dupn : string -> int -> string` that given a string `s` and an integer `n` concatenates `s` with itself `n` times.

**Example:** `dupn "Hi " 3 = "Hi Hi Hi "`

## Exercise 1.7 (based on 2.8 from HR)

The following figure gives the first part of Pascal's triangle

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
```

The entries of the triangle are called binomial coefficients. The  $k$ 'th binomial coefficient of the  $n$ 'th row is denoted  $\binom{n}{k}$ , for  $n \geq 0$  and  $k \leq n$ . For example,  $\binom{2}{1} = 2$  and  $\binom{4}{2} = 6$  (note that we start counting at 0). The first and last binomial coefficients, that is,  $\binom{n}{0}$  and  $\binom{n}{n}$ , of row  $n$  are both 1. Binomial coefficient inside a row is the sum of the two binomial coefficients immediately above it. These properties can be expressed as follows:

$$\binom{n}{0} = \binom{n}{n} = 1$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \text{ if } n \neq 0, k \neq 0, \text{ and } n > k$$

Declare a function `bin : int * int -> int` that given a pair  $(n, k)$  computes  $\binom{n}{k}$ .

### Extra-curricular activities

There is an [interesting video](#) on fractal-like properties that are found when looking at even and odd numbers of Pascal's triangle.

## Exercise 1.8

This assignment can be ported to the interpreter that you will write towards the end of the course.

Create a function `readInt` of type `unit -> int` that reads input from the console, line by line, until an integer is received at which point that integer is returned. For every input that is not an integer the function must print `<input> is not an integer`, where `<input>` is what the user entered into the console.

You may use the following two functions

```
... let readFromConsole () =  
  System.Console.ReadLine().Trim()  
... let tryParseInt (str : string) =  
  System.Int32.TryParse str
```

where `readFromConsole : unit -> string` reads one line from the console and `tryParseInt : string -> bool * int` takes an integer `str` as an argument and returns a tuple of the form `(success, result)` where `success` is a boolean

that is `true` if `str` is an integer, and `false` otherwise. Moreover, the variable `result` will contain the string translated to an integer when `success` is `true`.

Finally, use `printfn` to print strings and `+` to compose them. For instance, the command `printfn ("Mary had a little " + "llama")` prints `Mary had a little llama` to the screen.

## Yellow Exercises

### Exercise 1.9

Assume the time of day is represented as a pair `(hh, mm) : int * int` where `hh` represents the hour (a number between `0` and `23`) and `mm` represents the minutes (a number between `0` and `59`).

Write a function `timediff : int * int -> int * int -> int` so that `timediff t1 t2` computes the difference in minutes between `t1` and `t2`, i.e., `t2 - t1`.

#### Examples:

```
> timediff (12, 34) (11, 35);;  
val it : int = -59  
  
> timediff (12, 34) (13, 35);;  
val it : int = 61
```

### Exercise 1.10

Write a function `minutes : int * int -> int` that computes the number of minutes since

midnight.

**Hint:** This is easily done using the function `timediff`.

**Examples:**

```
> minutes (14,24);;  
val it : int = 864  
  
> minutes (23,1);;  
val it : int = 1381
```

## Red Exercises

### Exercise 1.11 (based on 2.13 from HR)

Write declarations for the functions `curry` and `uncurry`, which have the types

```
curry : ('a * 'b -> 'c) -> 'a -> 'b ->  
'c
```

```
uncurry : ('a -> 'b -> 'c) -> 'a * 'b -  
> 'c
```

and are defined as follows:

- The `curry` function takes a function `f` of type `'a * 'b -> 'c` as its first argument. This function `f` takes a single tuple `(a, b)`, where `a` and `b` have the types `'a` and `'b` respectively, and returns a value of type `'c`. The `curry` function then returns a function of type `'a -> 'b -> 'c` that given two arguments `a` and `b`, applies them to `f` and returns the result.
- The `uncurry` does the reverse. It takes a function `f` of type `'a -> 'b -> 'c` as its first

argument. This function  $f$  takes two arguments  $a$  and  $b$ , that have the types  $'a$  and  $'b$  respectively, and returns a value of type  $'c$ . The `uncurry` function then returns a function of type  $'a * 'b \rightarrow 'c$  that given a tuple  $(a, b)$  takes  $a$  and  $b$ , applies them to  $f$ , and returns the result.

More formally,

- $\text{uncurry } (\text{curry } f) (x, y) = f (x, y)$
- $\text{curry } (\text{uncurry } f) x y = f x y$

Write definitions of `curry` and `uncurry`.

### Examples:

```
> curry (fun (x, y) -> x + y) 5 3;;  
val it : int = 8  
  
> uncurry (fun x y -> x + y) (5, 3);;  
val it : int = 8
```